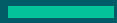




COURSE ON

Biological Data Analysis with R



What We'll Cover



PART 1

Introduction to R

Core language: syntax, data structures, functions, control flow, packages, and data wrangling — the foundations every analysis is built on.



PART 2

Biological Data Analysis

Applying R to real life-science problems: sequence data, gene expression, statistics, phylogenetics, and machine learning in biology.

What is R?

R is a free, open-source programming language and environment **purpose-built for statistical computing, data analysis, and graphics.**



Created in 1993

By Ross Ihaka and Robert Gentleman at the University of Auckland, as a dialect of the S language.



Package ecosystem

Over 20,000 packages on CRAN, plus Bioconductor for genomics and life-science data.



Built for statistics & visualization

Native support for statistical models, hypothesis tests, and publication-quality plots.

QUICK FACTS

- **Free & Open Source**
No license cost, fully reproducible
- **Cross-platform**
Windows, macOS, Linux
- **Interpreted language**
Run code line-by-line interactively
- **Strong community**
CRAN, Bioconductor, Stack Overflow, R-bloggers

Why R for Scientific & Biological Data?



Designed for analysis

Statistics, modeling, and hypothesis testing are first-class features, not add-ons.



Bioconductor ecosystem

2,300+ curated packages for genomics, transcriptomics, and proteomics.



Publication-ready graphics

ggplot2 and base graphics produce journal-quality figures directly from code.



Reproducible research

Scripts + R Markdown let every result be re-run, audited, and shared.

R in Context: How It Compares

Criterion	R	Python	Excel / SPSS
Statistical depth	Extensive, native	Good, via libraries	Limited
Bioinformatics tools	Bioconductor (2,300+)	Biopython (smaller)	Minimal
Graphics quality	Publication-ready (ggplot2)	Good (matplotlib/seaborn)	Basic charts
Learning curve	Moderate	Moderate	Low
Cost	Free	Free	Often licensed
Reproducibility	Excellent (scripts, R Markdown)	Excellent (notebooks)	Poor (manual steps)

R

STRENGTHS

- Free and Open Source
- Strong User Community
- Highly extensible, flexible
- Implementation of high end statistical methods
- Flexible graphics and intelligent defaults

Getting Set Up: R & RStudio

R is the engine — the language and its interpreter. **RStudio** is the most popular IDE built around it: a friendlier cockpit for writing, running, and debugging R code.

1

Download R

Get the base R distribution from CRAN (cran.r-project.org) for your operating system.

2

Install RStudio Desktop

Download the free RStudio Desktop IDE from posit.co — it detects R automatically.

3

Verify the install

Open RStudio and run `R.version.string` in the console to confirm R is linked correctly.

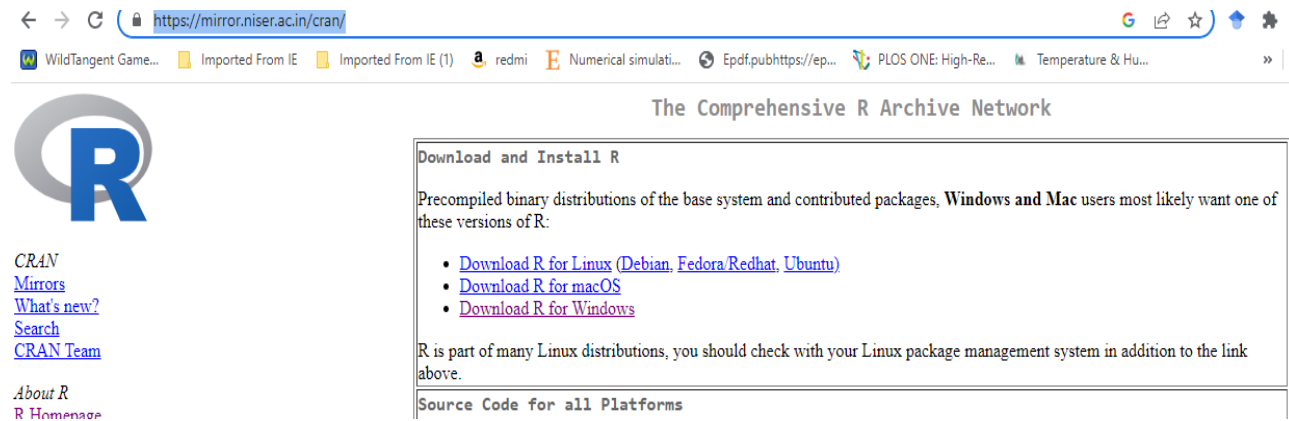
4

Install key packages

Run `install.packages("tidyverse")` to get essential data-analysis tools in one step.

R software installation

1. Go to <https://cran.r-project.org> or <https://mirror.niser.ac.in/cran/> and click on Download R for windows.

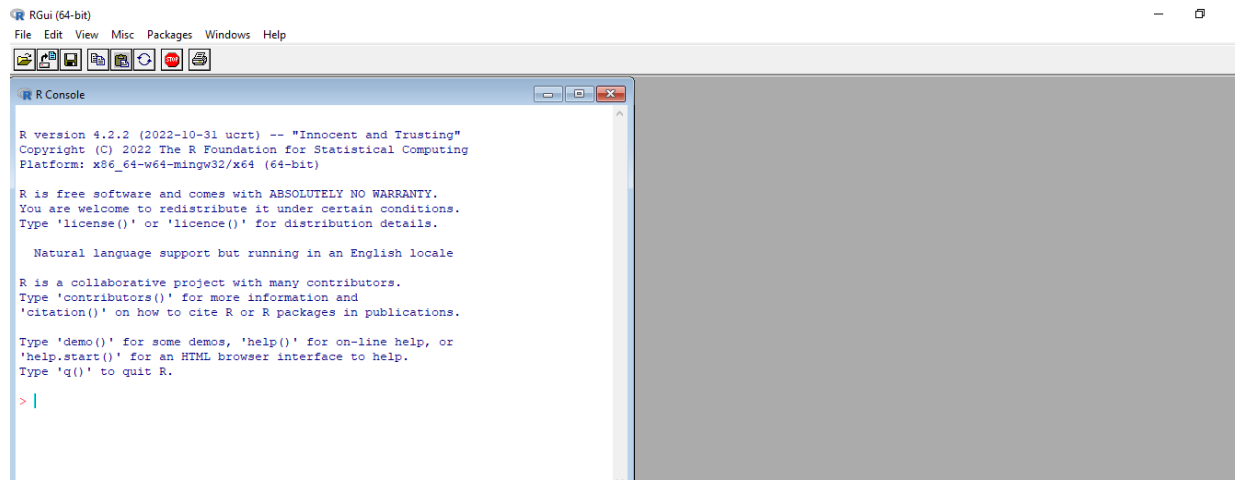


2. In the next page, click on latest version of R for Windows to download the installation file.

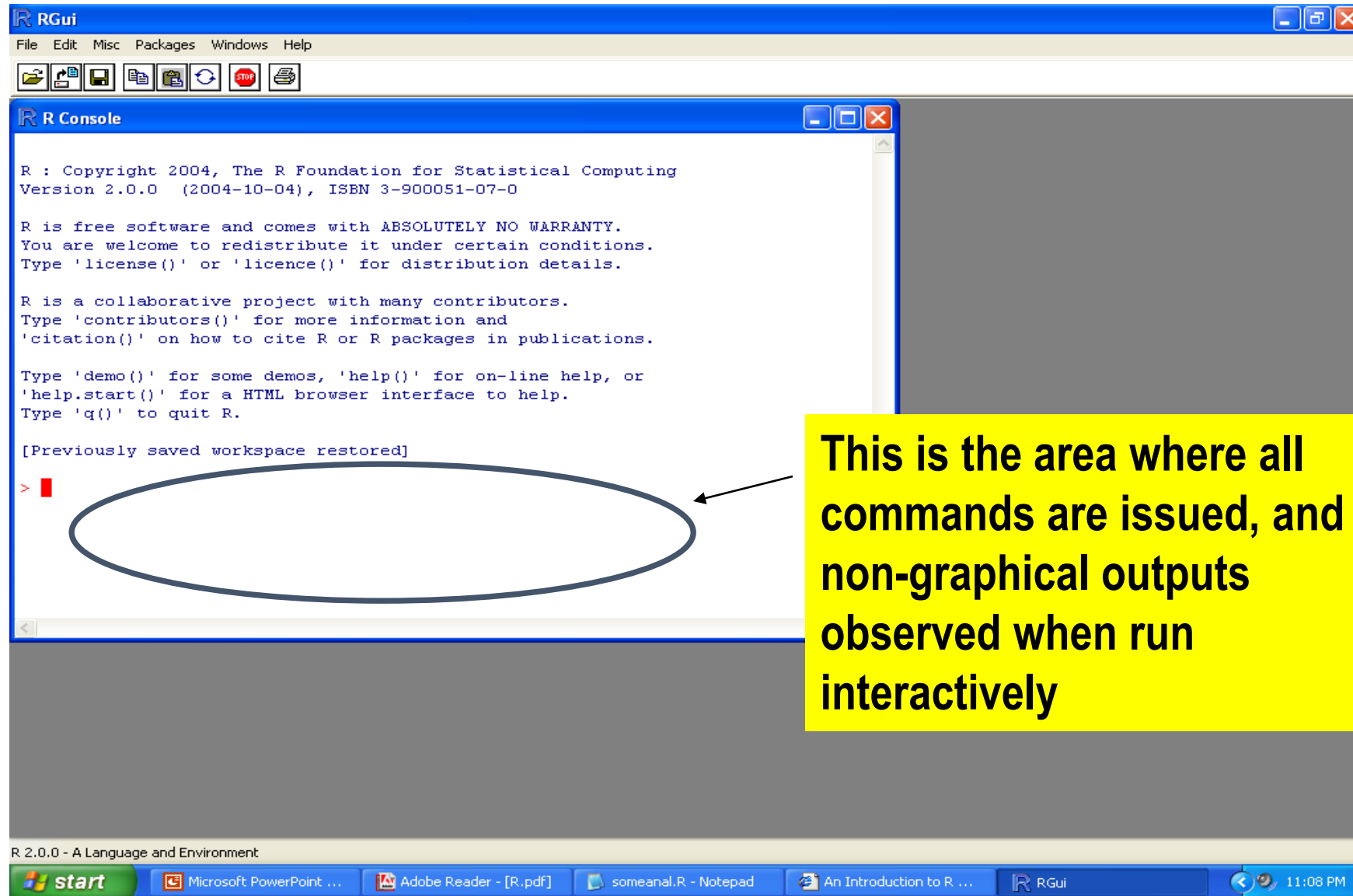
3. Install R software using downloaded file.

4. After installation, click on desktop shortcut  to open R programming software.

5. The R software opening window:



R Working Area



Perform simple descriptive statistics on small data set using R. For R commands, check the following screenshot and **practice it**.



```
RGui (64-bit) - [R Console]
File Edit View Misc Packages Windows Help

R version 4.2.2 (2022-10-31 ucrt) -- "Innocent and Trusting"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> data1 = c(13,16,30,32,40,18,20,21)
> data1
[1] 13 16 30 32 40 18 20 21
> mean(data1)
[1] 23.75
> median(data1)
[1] 20.5
> var(data1)
[1] 85.92857
```

_____ Add small dataset to R

_____ Load the added dataset

_____ Finding mean of dataset

_____ Finding median of dataset

_____ Finding sample variance of dataset

Tour of the RStudio Interface

1. Source Editor



Write and save .R scripts; run selected lines or the whole file.

2. Console



Executes code interactively and shows immediate output.

3. Environment / History



Lists active variables, data frames, and past commands.

4. Files / Plots / Packages / Help



Browse files, view plots, manage packages, and read documentation.

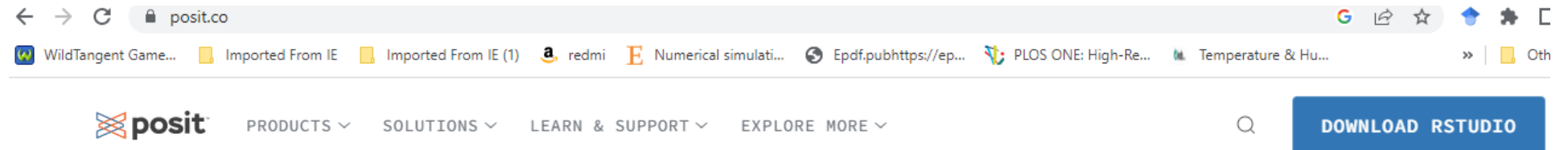
RStudio is

An Integrated Development Environment (IDE) for R

An easy (easier) way to use R

Available as a desktop product

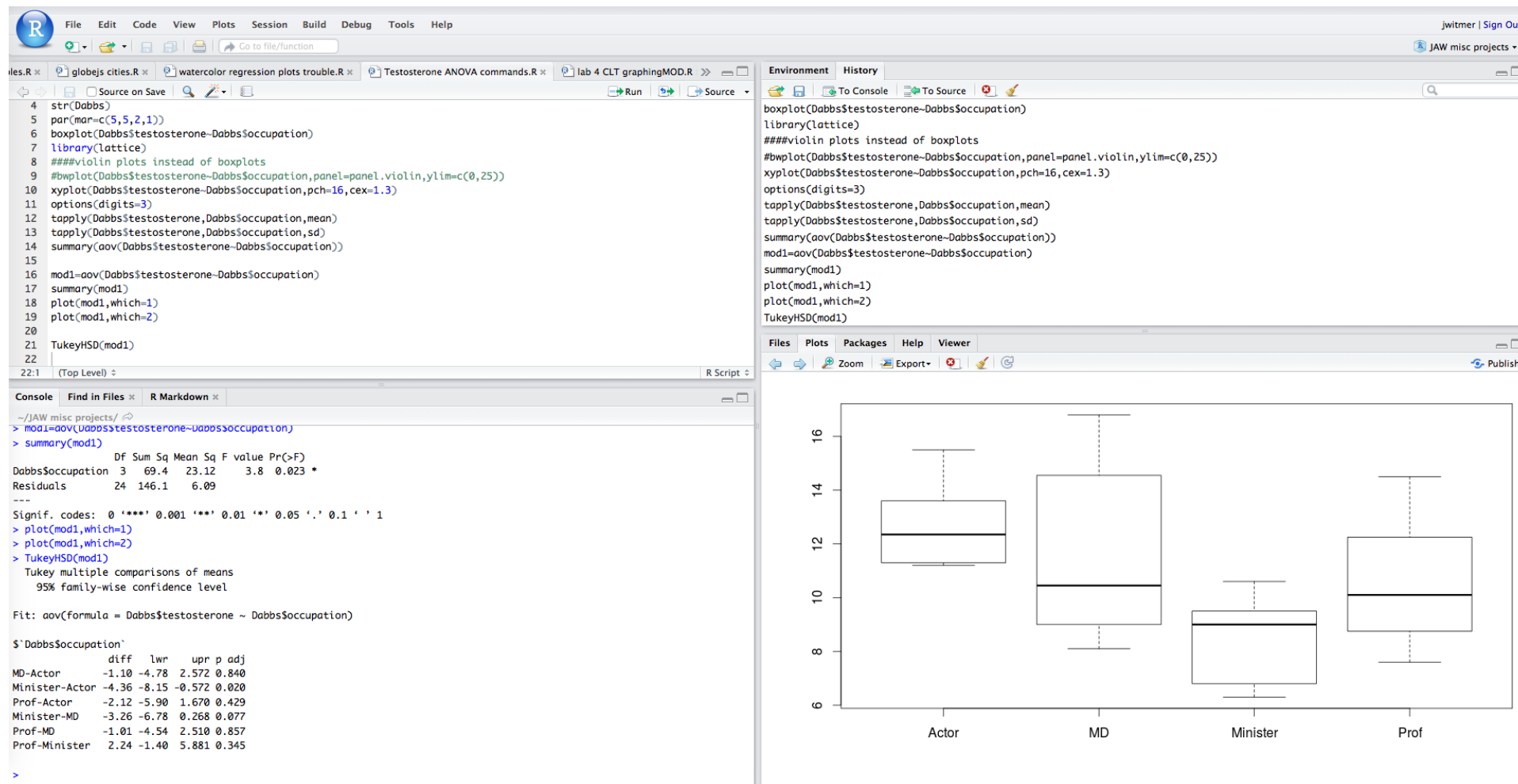
Free



RStudio is now Posit,
our mission continues

At Posit, our goal is to make data science more open, intuitive, accessible, and collaborative. We provide tools that make it easy for individuals, teams, and enterprises to leverage powerful

RStudio screen shot



Variables, Assignment & Data Types

Variables are created with the assignment operator `<-` (preferred) or `=`. R infers the type automatically.

```
# Assigning variables
gene_name <- "TP53"
expression_level <- 12.7
is_upregulated <- TRUE

# Checking the type
class(expression_level)
#> [1] "numeric"
```

CORE DATA TYPES

numeric

Decimal or whole numbers

12.7, 3, -0.5

character

Text strings

"TP53", "sampleA"

logical

TRUE / FALSE values

TRUE, FALSE

integer / factor

Whole numbers & categories

1L, 2L / levels: low, med, high

Data Structures in R



Vector

1D sequence, single data type

```
c(1, 2, 3, 4)
```



Matrix

2D grid, single data type

```
matrix(1:6, nrow=2)
```



List

Ordered collection, mixed types

```
list(id=1, name="A")
```



Data Frame

Table with typed columns

the workhorse for datasets

Data frames are central to biological work — each row a sample, each column a variable (e.g., gene, treatment, expression value).

```
samples <- data.frame(  
  id = c("S1", "S2", "S3"),  
  treatment = c("Control", "Drug", "Drug"),  
  expression = c(5.2, 8.9, 9.4)  
)
```

Operators & Built-in Functions

OPERATORS

Arithmetic

+ - * / ^ %%

Comparison

== != < > <= >=

Logical

& | ! && ||

Assignment

<- = <<-

COMMON BUILT-IN FUNCTIONS

```
mean(expression)      # average
sd(expression)         # std. deviation
length(samples)        # count
summary(samples)       # quick overview
sum(x); max(x); min(x)
paste("Gene", 1:3)     # string join
```

Functions follow the pattern `function_name(arguments)` — R has thousands built in, and thousands more via packages.

Control Structures: Conditions & Loops

IF / ELSE

```
if (expression_level > 8) {  
  print("Upregulated")  
} else if (expression_level < 3) {  
  print("Downregulated")  
} else {  
  print("No significant change")  
}
```

FOR LOOP

```
for (gene in gene_list) {  
  cat("Processing:", gene, "\n")  
}
```

WHILE LOOP

```
i <- 1  
while (i <= length(samples)) {  
  print(samples[i])  
  i <- i + 1  
}
```

Why it matters for biology

Loops and conditionals let you automate repetitive tasks — e.g., flagging every gene above a fold-change threshold, or processing hundreds of sample files without manual repetition.

Writing Your Own Functions

Custom functions package up logic you'll reuse — turning a 10-line analysis into a single, readable call.

```
# Define a function to classify gene expression
classify_expression <- function(value, threshold = 8) {
  if (value > threshold) {
    return("Upregulated")
  } else if (value < (threshold / 2)) {
    return("Downregulated")
  } else {
    return("Stable")
  }
}

# Call the function
classify_expression(9.4)
#> [1] "Upregulated"
```

Arguments

Inputs the function accepts, with optional defaults (threshold = 8).

Return value

The output sent back with return(), usable in further code.

Reusability

Write once, apply to thousands of genes or samples.

Packages: Extending R's Power

Packages are collections of functions, data, and documentation that extend base R. Two major repositories serve very different needs:



CRAN

Comprehensive R Archive Network

20,000+ general-purpose packages — data wrangling (dplyr), visualization (ggplot2), modeling, reporting.

```
install.packages("tidyverse")
```



Bioconductor

Specialized life-sciences repository

2,300+ packages for genomics, transcriptomics, proteomics, and sequence analysis, with curated annotation data.

```
BiocManager::install("DESeq2")
```

Importing Data & Basic Wrangling

READING & WRITING DATA

```
# Read a CSV file
data <- read.csv("expression_data.csv")

# Read Excel / other formats
library(readxl)
data <- read_excel("samples.xlsx")

# Write results back out
write.csv(results, "results.csv")
```

DATA WRANGLING WITH dplyr

```
library(dplyr)

data %>%
  filter(treatment == "Drug") %>%
  group_by(gene) %>%
  summarise(mean_expr = mean(expression))
```

The pipe operator %>% (or native |>)

Chains steps left-to-right — filter, then group, then summarise — instead of nesting functions, keeping analysis code readable top to bottom.

Data Visualization: Base R & ggplot2

BASE R PLOTTING

```
plot(x, y, main = "Expression vs Dose",  
     xlab = "Dose", ylab = "Expression")  
  
hist(expression)  
boxplot(expression ~ treatment)
```

Quick and dependency-free — ideal for fast, exploratory checks.

GGPLOT2: THE GRAMMAR OF GRAPHICS

```
library(ggplot2)  
  
ggplot(data, aes(x = treatment, y = expression)) +  
  geom_boxplot(fill = "#02C39A") +  
  labs(title = "Expression by Treatment") +  
  theme_minimal()
```

Layered syntax (data + aesthetics + geometry) produces polished, publication-ready figures — the standard for biological plots.

PART 2

Applications in Biological Data Analysis

From sequences to statistics: how R powers real life-science research pipelines



The Bioconductor Ecosystem

Bioconductor is a curated repository of R packages, annotation data, and workflows purpose-built for high-throughput biological data.



Genomics

Biostrings, GenomicRanges —
sequence manipulation, variant
analysis.



Transcriptomics

DESeq2, edgeR, limma — differential
gene expression from RNA-seq.



Proteomics

MSnbase, xcms — mass-spectrometry
data processing.



Pathway analysis

clusterProfiler, KEGGREST —
functional enrichment & pathways.

2,300+ packages, updated twice yearly, each with mandatory documentation, vignettes, and unit tests — a major reason Bioconductor tools are trusted in peer-reviewed research.

Sequence Analysis with R

SEQUENCE ANALYSIS

- **Reading sequences**

Import FASTA/FASTQ files with the Biostrings package.

- **Sequence manipulation**

Reverse-complement, translate, and search for motifs.

- **Alignment**

Pairwise or multiple sequence alignment (Biostrings, msa).

- **GC content & composition**

Quantify base composition across a genome or region.

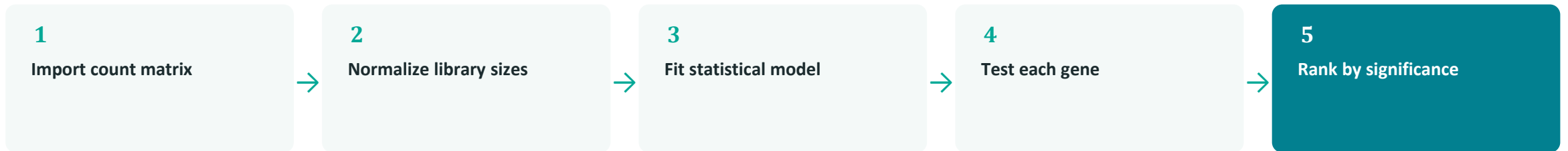
```
library(Biostrings)

seq <- readDNAStringSet("gene.fasta")
reverseComplement(seq)
letterFrequency(seq, letters = "GC")
translate(seq) # DNA -> protein
```

Used for: primer design, mutation screening, comparative genomics, and annotating regions of interest across species.

Gene Expression Analysis (RNA-seq)

A typical differential expression workflow moves from raw counts to a ranked, statistically tested gene list:



```
library(DESeq2)

dds <- DESeqDataSetFromMatrix(countData = counts,
                              colData = sample_info,
                              design = ~ treatment)

dds <- DESeq(dds)
results(dds, alpha = 0.05) # differentially expressed genes
```

Output: a table of genes with log2 fold-change and adjusted p-values — the starting point for pathway and functional analysis.

Statistical Analysis & Hypothesis Testing

Function	Purpose	Example use in biology
<code>t.test()</code>	Compare means	Control vs. treated group expression
<code>aov()</code> / ANOVA	Compare 3+ groups	Multiple drug-dose responses
<code>cor.test()</code>	Test association	Gene co-expression relationships
<code>chisq.test()</code>	Categorical association	Genotype vs. phenotype frequency
<code>shapiro.test()</code>	Normality check	Validate assumptions before testing

```
result <- t.test(expression ~ treatment, data = samples)
result$p.value    # e.g. 0.0031  -> statistically significant
```

Phylogenetic Analysis

Packages like ape, phangorn, and treeio let R build, manipulate, and visualize evolutionary trees directly from sequence alignments.



Distance calculation

Compute genetic distances between aligned sequences.



Tree construction

Build trees via neighbor-joining, maximum likelihood, or Bayesian methods.



Visualization

Plot and annotate trees — clades, bootstrap values, tip labels.

```
library(ape)
tree <- nj(dist.dna(dna_alignment))      # neighbor-joining tree
plot(tree, main = "Phylogenetic Tree")
add.scale.bar()
```

Visualizing Biological Data



Heatmaps

`pheatmap / ComplexHeatmap`

Cluster samples & genes by expression pattern.



Volcano plots

`ggplot2 + ggrepel`

Fold-change vs. significance in one view.



PCA plots

`prcomp() + ggplot2`

Reveal sample groupings and batch effects.



Network plots

`igraph`

Gene interaction & co-expression networks.

Microbiome & Ecological Data Analysis

Packages such as phyloseq and vegan bring community-ecology statistics to microbiome and environmental datasets.

- **Diversity indices**

Shannon, Simpson — quantify richness and evenness within a sample.

- **Ordination (NMDS/PCoA)**

Visualize similarity between microbial communities in 2D.

- **Beta diversity testing**

PERMANOVA to test whether groups differ significantly.

- **Taxonomic composition plots**

Stacked bar charts of relative abundance by taxon.

```
library(vegan)

diversity(otu_table, index = "shannon")
adonis2(otu_table ~ treatment, data = metadata)
ordination <- metaMDS(otu_table)
plot(ordination)
```

Relevant to soil, gut, wastewater, and constructed-wetland microbial community studies alike.

Machine Learning in Biology

R's caret, randomForest, and tidymodels packages bring predictive modeling to classification and biomarker discovery problems.



Disease classification

Predict diagnosis from gene expression or clinical features.



Feature/biomarker selection

Identify the genes most predictive of an outcome.



Model validation

Cross-validation and ROC/AUC to assess accuracy honestly.

```
library(randomForest)

model <- randomForest(diagnosis ~ ., data = train_set, ntree = 500)
predictions <- predict(model, newdata = test_set)
importance(model)    # which genes matter most
```

Resources for Further Learning



Learn the Language

- R for Data Science (Wickham & Grolemund) — free online
- swirl — learn R interactively inside R itself
- RStudio Education (education.rstudio.com)



Biology-Specific

- Bioconductor.org — package vignettes & workflows
- "Bioinformatics Data Skills" (Vince Buffalo)
- Galaxy Project & rOpenSci community tutorials



Community & Practice

- Stack Overflow / Bioconductor Support forum
- R-bloggers.com — applied examples & case studies
- Kaggle biological datasets for practice



Thank You